Question 1:

1. Any program in L1 can be transformed into an equivalent program in L11. To do this, take any variable that is defined in a define expression, and replace it with the expression itself.
2. Any program in L2 can be transformed into an equivalent program in L21. This is because we can replace variable bindings with lambda applications. In addition, to transform recursion calls we can use y-combinatorics (which wasn't taught yet but mentioned in the lecture notes.
3. Any program in L2 can be transformed into an equivalent program in L22. This is because in lambda expressions, only the last expression matters anyway, and as for the parameters, you can use nested functions that each only take one parameter.
4. Any program can be transformed from L2 to L23. This is because in both these languages all the information is known while the program is being written. Since they are functional, they do not support receiving external output from the user during the program execution. Therefore, since the programmer knows which lambdas are defined in the program, they can instead implement the body of the lambdas directly in the program expression.

5. Special forms are required in programming languages for logic that is evaluated in a different way then the default evaluation way. If define was evaluated the same way as a primitive, the var binding name would be evaluated before the expression, but that is not how the logic of defining a variable works. For example the program (define x 3) where define is a primitive op would try to calculate the value of x. This would crash because x has no value.
6. Since a purely functional language has no side effects, a function body with multiple expressions is not required. However, it is useful in L2, because it is a comfortable way to scope. In L3, the special form let is added, therefore multiple expressions are no longer useful
7. Lexical addresses are a way to determine which variable declaration a variable reference is bound (unambiguously). It is defined as a tuple : [var: depth pos]
   var: the name of the variable
   depth: The number of nested scopes (lambdas) between the variable's use and its declaration
   pos: The position of the variable in the relevant scope

Example:
In the program:
(lambda (a) (lambda (b) (+ a b)))
the lexical addresses are: [a:1,0] [b:0,0]

8. In the PrimOp representation, the primitive operations are recognized during parsing, this gives the advantage of allowing the interpreter to execute these operations quicker.
However, in the Closure representation, primitives are saved as varDec in the GE, therefore you can add a new primitive operation without changing the code of the interpreter, the applyProcedure remains the same, only a new varDec must be added.

9.
   a) One should switch from from applicative order to normal order when there might be a bug in one of the arguments of a function (such as a runtime error or an infinite loop) or when the use of one the arguments is not sure.
   For example, in the program:
   (define f (lambda (a b) a))
   (f 5 (/ 1 0)
   The function body doesn't require calculating the operand b, and evaluating it in applicative order is unnecessary and will cause a runtime error.
   b) One should switch from normal order to applicative order when there is a repeated use of an argument in the body of a function, especially when the evaluation of the operand is a heavy computation.
   For example, in the program:
   (lambda (x) (+x (+x (+x x)))(+1 (+2 (+3 (+4 5)))))
   to evaluate in normal order you would have to evaluate (+1 (+2 (+3 (+4 5)))) 4 times whereas in applicative order you would only have to evaluate it once, making the program much more efficient

10.
   a) When we evaluate a function call, the L3 interpreter first evaluates the function parameters. However, before evaluating the body of the function, the interpreter substitutes the operands with their evaluated value. However, the substitution function receives a LitExp and not a

Value type. Therefore, the purpose of valueToLitExp is to take the calculated operands and make them into literal expressions

b) In the normal evaluation strategy we evaluate the operands only inside the method body evaluation, therefore we substitute arguments that are not evaluated yet, therefore there is no need to translate values into literal expressions. This is why valueToLitExp is not needed
c) In the environment model, we use lexical scoping, therefore substitution is also not necessary. Since there is no substitution, there is also no need to translate values into LitExp therefore valueToLitExp is not needed

11. The purpose of renaming is to prevent a logical error in the substitution model known as variable capture. This happens when an expression containing a free variable is substituted into a function body that happens to have a bound parameter with the exact same name, causing the free variable to be accidentally trapped by the inner scope. Therefore before substituting, the interpreter renames variables so no argument is named the same as a free variable used in the body.

a) Therefore, in the environment model, where we don't substitute, but instead use a lexical address. This way the interpreter can determine which scope each variable belongs to so variable capture cannot happen, so renaming is not necessary.
b) In a case where a term in the substitutional model is closed, variable capture cannot occur. This is why renaming in not needed in this case

Question 2:

Expression List:

```
(define pi 3.14)
(define square (lambda (x) (* x x)))
(define circle
        (lambda (x y radius)
                (lambda (msg)
                        (if (eq? msg 'area)  (* (square radius) pi)
                                (if (eq? msg 'perimeter) (* 2 pi radius)
                                'error
                                )
                        )
                )
        )
)
```

```
(define c (circle 0 0 3))
(c 'area)
```

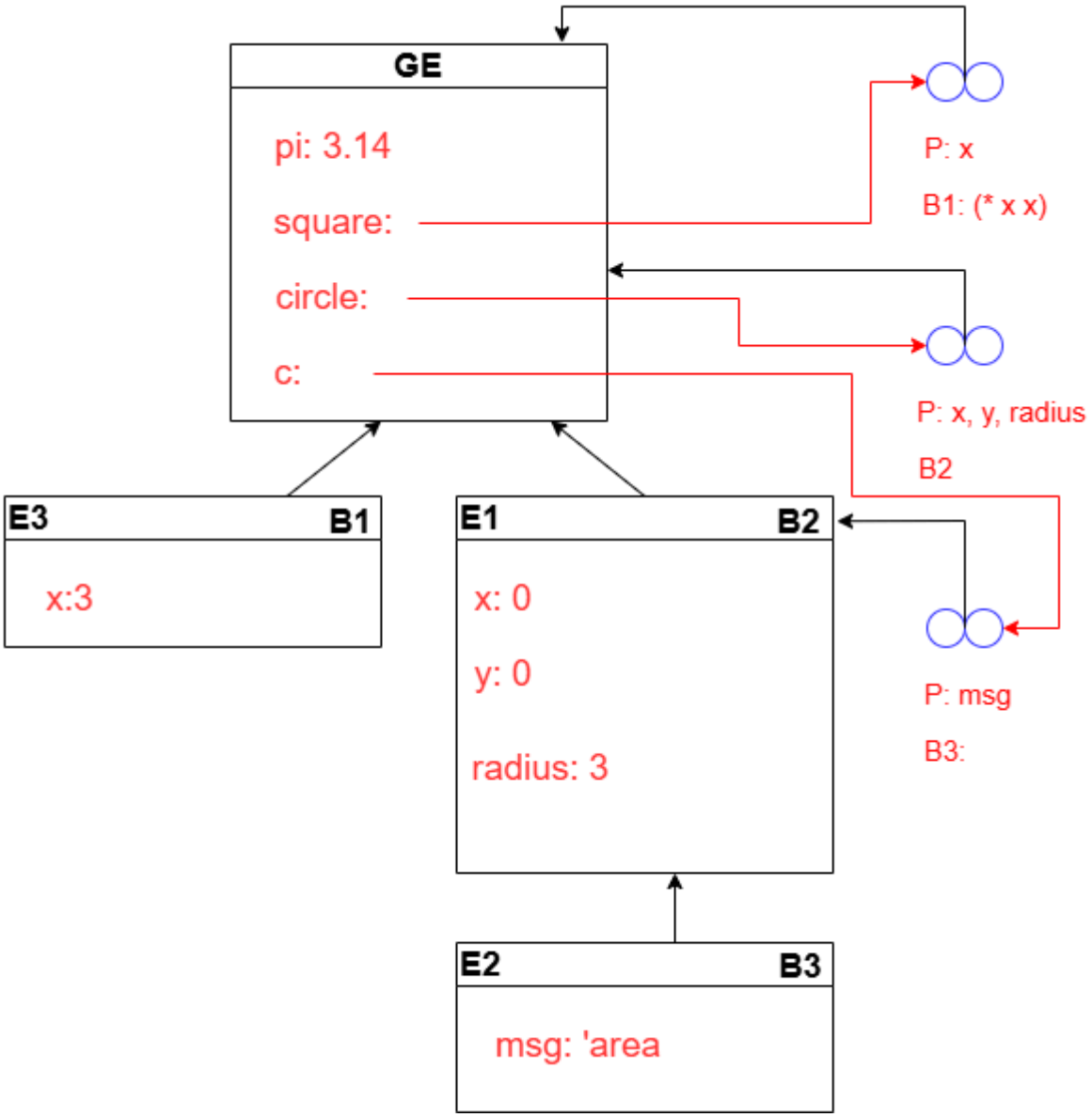
List of expression to l3ApplicativeEval:

```
-3.14
-(lambda (x) (* x x))
- (lambda (x y radius)
                (lambda (msg)
                        (if (eq? msg 'area)  (* (square radius) pi)
                                (if (eq? msg 'perimeter) (* 2 pi radius)
                                'error
                                )
                        )
                )

        )
- (circle 0 0 3)
-circle
-0
-0
-3
-(lambda (msg)
        (if (eq? msg 'area)  (* (square 3) pi)
                (if (eq? msg 'perimeter) (* 2 pi 3)
                'error
                )
        )
)
-(c 'area)
-c
-'area
-(lambda ('area)
        (if (eq? 'area 'area)  (* (square 3) pi)
                (if (eq? 'area 'perimeter) (* 2 pi 3)
                'error
                )
        )
)
-(eq? 'area 'area)
-eq?
```

-’area
-’area
-(* (square 3) pi)
-*
-(square 3)
-square
-3
(* 3 3)
-*
-3
-3
-pi

Environment diagram:



* B2: (lambda (msg)
(if (eq? msg 'area) (* (square radius) pi)
(if (eq? msg 'perimeter) (* 2 pi radius)
‘error
)
)
)
* B3: (if (eq? msg 'area) (* (square radius) pi)

```
        (if (eq? msg 'perimeter) (* 2 pi radius)
        'error
        )
)
```